

Headless Digital Identity, KYC, Lending & Payments Platform

Consolidated High-Level & Low-Level Design Document

Salesforce · MuleSoft · Sarvam AI · Anthropic Claude · PhonePe

Document type	Consolidated HLD / LLD — single source of truth
Scope	Digital identity & KYC, loan origination & servicing, payments, document generation & management, AI retrieval
Frontend	Salesforce Experience Cloud (Lightning Web Components)
Orchestration	MuleSoft Anypoint (Mule 4.6)
AI services	Sarvam AI (Vision OCR, STT, Language-ID, Translate, Transliterate, TTS, Face-verify*); Anthropic Claude (consent, adjudication, liveness, grounded answers)
Payments	PhonePe Merchant (collections, disbursement, refunds, NACH/eNACH debit)
System of record	Salesforce Headless 360 — Apex REST + Visualforce document rendering
Data model	15 custom objects across identity, lending, payments, documents and knowledge
Intelligence	Document RAG (retrieval + citations) and an MCP tool surface
Status	Build-aligned engineering baseline

Table of Contents

PART A

Platform Overview

1. Purpose & Business Context

This document is the consolidated design for a headless digital platform that takes an Indian citizen from identity verification through to a fully serviced, insured loan — entirely through API orchestration, with Salesforce as the system of record and front door. It unifies four capability layers into one coherent architecture: (1) multi-channel identity and KYC; (2) loan origination, underwriting and servicing; (3) payments and mandates through a PhonePe merchant integration; and (4) regulated document generation, document management, and AI-assisted retrieval over those documents.

The platform is “headless” in that every capability is exposed as a service: the Experience Cloud portal is a thin presentation layer, MuleSoft is the orchestration spine that fans out to Sarvam AI, Claude, PhonePe, eSign and UIDAI, and Salesforce Apex REST is the transactional core that commits records and renders documents. Any channel — web, mobile, partner, or field-agent device — can drive the same services.

1.1 Capability layers

Layer	What it does	Primary outputs
Identity & KYC	Verify identity from document, video liveness, or Aadhaar offline; capture multilingual consent; adjudicate with AI	Verified application + bilingual KYC report
Lending	Originate, underwrite, price, book and amortise loans; schedule EMIs	Loan application, loan account, repayment schedule
Payments	Collect EMIs, disburse, refund and run mandate debits via PhonePe	Payment transactions, NACH/eNACH mandates
Documents	Generate KFS, agreements, mandates and insurance documents; eSign; store and version	Generated documents in a managed store
Intelligence	Retrieve answers grounded in the borrower's own documents; expose safe tools	RAG answers with citations; MCP tools

1.2 Personas

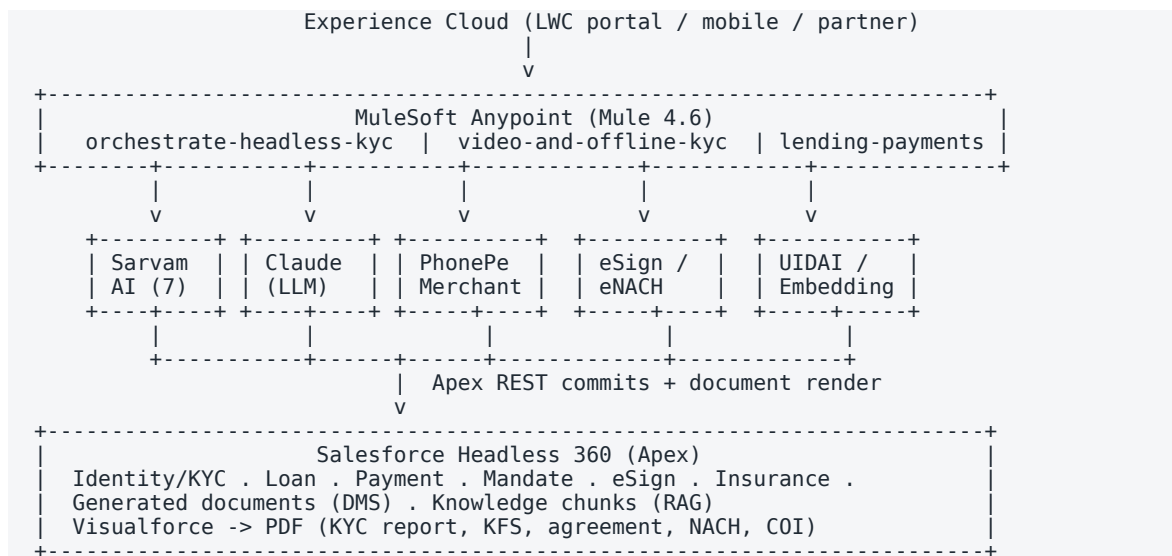
The platform serves distinct human and system actors. Each persona drives a defined set of surfaces and APIs, and the AI tool surface is scoped per persona (Section 22).

Persona	Responsibilities	Key surfaces / APIs
Applicant / Borrower	Onboards, accepts terms, signs, and self-serves on the loan	KYC submit, video, KFS & agreement review, eSign, ask-documents
Field Agent	Onboards applicants in person, often offline	Offline Aadhaar capture, agent-sync
KYC / Compliance Reviewer	Reviews escalated video sessions; signs off KYC	Agent review of escalated sessions, KYC report
Credit / Underwriting Officer	Reviews referred applications; issues documents	Underwriting decision, document generation, eSign initiate
Collections Officer	Works upcoming and overdue instalments	Collections queue, retry-debit (PhonePe)

Persona	Responsibilities	Key surfaces / APIs
Operations Manager	Owns the book; monitors KPIs and the disbursement queue	Portfolio summary, disbursement queue, ask-portfolio
System / Integration	MuleSoft + scheduled jobs driving callbacks, debits, payouts	All Apex REST endpoints (service account)

2. End-to-End Architecture

All client traffic enters through Salesforce Experience Cloud. MuleSoft mediates every external dependency; secrets live only in MuleSoft secured properties and a Salesforce Named Credential. Salesforce Headless 360 holds the data model and renders documents as PDFs.



2.1 Component inventory

Tier	Components
Presentation	Experience Cloud site; LWC onboarding/loan/document widgets; operations servicing dashboards (collections, disbursement queue, ask-portfolio)
Orchestration (MuleSoft)	orchestrate-headless-kyc · video-and-offline-kyc · lending-payments-docs flows; secured property store; API-led request configs
Apex services	HeadlessKYCService, VideoKYCService, OfflineKYCService, KYCReportService, LoanManagementService, LoanServicingService, PhonePePaymentService, NachMandateService, ESignService, DMSService, LoanDocumentService, DocumentRAGService (+ controllers)
Document rendering	Visualforce PDF pages: KYC report, KFS, loan agreement, NACH mandate, insurance COI
Data	15 custom objects (Section 23); Salesforce Files for PDFs and photos
External	Sarvam AI; Anthropic Claude; PhonePe merchant; eSign/eNACH provider; UIDAI; embedding + vector store; MCP server

3. Design Principles

- **Headless & API-led:** each capability is a service; the UI is replaceable and channels are interchangeable.

- **Single system of record:** Salesforce owns the data model and the audit trail; MuleSoft holds no business state.
- **Secrets isolation:** API keys and tokens live only in MuleSoft secured properties and a Salesforce Named Credential, never in the client.
- **AI with a human backstop:** Claude adjudicates and reasons, but weak or risky outcomes escalate to a human; sensitive actions are never fully autonomous.
- **Multilingual by default:** consent, confirmations and citizen documents are produced in the applicant's own language.
- **Document-centric compliance:** every regulated step yields a versioned, retained document in a managed store.

PART B

Identity & KYC

4. KYC Channels

Identity can be proven through three complementary channels. Document is the baseline; Video and Offline are additive assurance channels chosen per application. Each channel is an independent MuleSoft flow that writes to its own Salesforce object, and the application's overall status reflects the combination.

Channel	Method	Salesforce object	AI involvement
Document	ID capture + multilingual audio & written consent	KYC_Application__c + Consent/Sarvam/Claude	OCR, translation, consent validation, adjudication
Video	Self-recorded liveness + spoken challenge, agent escalation	Video_KYC_Session__c	STT, face-verify, liveness adjudication
Aadhaar offline	UIDAI offline e-KYC ZIP + share code	Offline_KYC_Record__c	Signature verification (deterministic)
Field agent	Offline capture synced on reconnect	Offline_KYC_Record__c	—

5. Document KYC & Multilingual Consent

The applicant submits an identity document and provides consent twice: a spoken statement in any major Indian language and a typed statement. Sarvam Vision OCR extracts the document fields; Language-ID detects the spoken language; Translate and Transliterate normalise names and address to English while preserving the native script; Claude validates that consent is informed and unambiguous; a TTS confirmation is produced in the applicant's language.

5.1 Sarvam operations used

Operation	Endpoint / model	Role
Vision OCR	POST /v1/document	Extract identity-document fields
Speech-to-Text	POST /speech-to-text · saaras:v3	Transcribe spoken consent / challenge
Language-ID	POST /text-lid	Detect the applicant's language
Translate	POST /translate	Normalise consent & address to English
Transliterate	POST /transliterate	Romanise the native-script name
Text-to-Speech	POST /text-to-speech · bulbul:v2	Localized spoken confirmation
Face-verify*	POST /v1/face-verify	Liveness + face match (provider-abstract)

6. Video KYC — Liveness with Agent Escalation

The applicant records a short video reading a randomized on-screen code. Sarvam STT transcribes the spoken code; the face-verify service returns a liveness score and a face-match score against the

document photo; Claude adjudicates the combination and decides whether a human agent is needed.

```
Boolean weak = (livenessScore == null || livenessScore < 0.80)
              || (faceMatchScore == null || faceMatchScore < 0.75)
              || (challengeMatch != true);
if (agentEscalated || weak) {
    session.Session_Status__c = 'Escalated';
    application.Status__c      = 'Pending Agent Review';
} else {
    session.Session_Status__c = 'Passed';
}
```

Escalated sessions are worked by a human verifier; on approval the application proceeds to report generation.

7. Offline KYC — Aadhaar XML & Field Agent

- **Aadhaar offline e-KYC:** the resident's password-protected ZIP is opened with the 4-character share code; the embedded XML is signature-verified against the UIDAI certificate; demographics and the Pht (base64 photo) element are extracted.
- **Field-agent capture:** a capture taken offline on an agent device is queued and synced through a dedicated endpoint when connectivity returns.

The applicant photo from any channel is stored as a Salesforce File and linked to the metadata so the report can embed it.

8. AI Adjudication

Claude (claude-sonnet-4-5) is called for consent validation and a cross-modal adjudication that weighs document fields, consent, and any video/offline signals into a verdict — Pass, Escalated, or Reject — with a risk score, a cross-modal match score, a policy-reasoning summary and an auditor narrative. The narrative is persisted for audit and reused in the report.

9. Bilingual KYC Report

On conclusive verification, Apex renders a Visualforce PDF containing identity details, a per-channel verification summary, consent records, the compliance decision and the auditor narrative — each key section labelled in both English and the applicant's detected language — with the verified photograph embedded. The PDF is stored as a Salesforce File via the document store (Section 17).

```
PageReference pdf = Page.KYCBilingualReport;
pdf.getParameters().put('id', kycApplicationId);
Blob body = Test.isRunningTest() ? Blob.valueOf('TEST') : pdf.getContentAsPDF();
// stored as ContentVersion + ContentDocumentLink; photo embedded as a base64 data URI
```

Indic fonts: the page embeds a Unicode font via @font-face on a static resource so Devanagari/Tamil/etc. render in the PDF (ship a real Noto TTF or enable Advanced PDF rendering).

PART C

Loan Management

10. Loan Lifecycle

A verified applicant can apply for a loan. The application moves through a controlled status machine as it is underwritten, priced, documented, mandated and disbursed; on disbursement a loan account is booked and an amortised repayment schedule is generated.

```
Draft -> Submitted -> Underwriting -> Approved -> KFS Issued
      -> Agreement Signed -> Mandate Active -> Disbursed
                        \-> Rejected
```

10.1 Loan objects

Object	Holds
Loan_Application__c	Requested/approved amount, tenure, rate, EMI, processing fee, APR, purpose, credit score, underwriting decision & notes, status
Loan_Account__c	Booked loan: principal, outstanding, disbursed amount/date, rate, EMI, tenure, next due date, status
Repayment_Schedule__c	Per-installment due date, EMI, principal & interest components, outstanding-after, paid date, linked payment, status

11. Underwriting & Pricing

Underwriting combines a credit score with Claude's KYC verdict and a recommended amount and rate. `LoanManagementService.underwrite` records the decision and notes; on approval it computes the EMI and APR.

11.1 EMI (reducing balance)

```
Decimal r = (rateAnnualPct / 100) / 12;           // monthly rate
Double rp = Math.pow(1 + (Double)r, (Double)months);
Decimal emi = principal * r * Decimal.valueOf(rp) / (Decimal.valueOf(rp) - 1);
```

11.2 APR (fee-inclusive)

```
Decimal totalPaid = emi * months;
Decimal totalCost = totalPaid - principal + (fee == null ? 0 : fee);
Decimal apr = (totalCost / principal) / (Decimal.valueOf(months) / 12) * 100;
```

12. Booking & Amortisation

On activation, `LoanManagementService.activate` creates the `Loan_Account__c` and builds the schedule: each installment splits the EMI into interest ($\text{outstanding} \times \text{monthly rate}$) and principal ($\text{EMI} - \text{interest}$), reduces the outstanding, and is dated to the 5th of each month. Disbursement is recorded against the account, and the first due date is set.

```
for (Integer i = 1; i <= months; i++) {
    Decimal interest = (outstanding * monthlyRate).setScale(2);
    Decimal principalComp = (emi - interest).setScale(2);
    outstanding -= principalComp;
    // Repayment_Schedule__c row: due 5th of month i, components, outstanding-after
}
```


12A. Loan Servicing & Collections

Once loans are booked, the operations team services the book through read-only portfolio APIs exposed by LoanServicingService (REST /v1/portfolio/*). These power the servicing dashboards and the ask-portfolio assistant; no financial action is taken in this layer.

12A.1 Portfolio reads

Endpoint	Returns
GET /v1/portfolio/summary	Book KPIs: active loans, total outstanding, overdue count & amount, collections due today, disbursed month-to-date, loans awaiting disbursement
GET /v1/portfolio/collections	Upcoming (next 30 days) + overdue instalments with borrower, loan, due date, amount and days overdue
GET /v1/portfolio/disbursements	Loans that are signed/mandated but not yet disbursed — the payout queue

12A.2 Collections

Collections distinguishes **overdue** (a pending instalment past its due date, with days-overdue computed) from **upcoming** (due within 30 days). A collections officer can send a reminder or retry the mandate debit, which calls POST /v1/phonepe/initiate as a UPI-mandate collection; a successful callback marks the instalment Paid and reduces the outstanding balance.

```
// overdue: Status IN ('Pending','Overdue') AND Due_Date < TODAY
// upcoming: Status = 'Pending' AND Due_Date <= TODAY + 30
// retry-debit -> /v1/phonepe/initiate (type 'EMI Collection', UPI mandate)
```

12A.3 Disbursement queue & dashboards

The disbursement queue lists applications in Agreement Signed or Mandate Active; those with an active mandate are ready to pay out. Disbursing calls the PhonePe payout path (Section 19), and the success callback books the loan. An operations console renders three views over these reads — collections (overdue / upcoming), the disbursement queue, and an ask-portfolio assistant — with a six-tile KPI header (active loans, outstanding, overdue, due today, disbursed MTD, awaiting disbursement).

PART D

Documents, Agreements & DMS

13. Document Set

Every regulated step produces a document. Documents are rendered from Visualforce as PDFs, stored in the document management store, and (where required) signed.

Document	When	Page / source
KYC report	KYC conclusively verified	KYCBilingualReport
Key Facts Statement (KFS)	Loan approved	KFSDocument
Loan agreement	Borrower accepts terms	LoanAgreement (+ eSign)
NACH / eNACH mandate	Repayment setup	NACHMandateForm
Insurance proposal / COI	Credit-life cover	InsuranceCOI
Sanction / welcome / foreclosure letters	Lifecycle events	Generated via DMS

14. Key Facts Statement (KFS)

An RBI-style bilingual KFS summarising amount, tenure, rate, EMI, fees, APR and a sample schedule, with penal-charge, cooling-off and grievance notes. Rendered by `KFSDocument` through `LoanDocumentController`, which supplies borrower, schedule (first six installments), mandate and insurance context plus bilingual labels. Issuing the KFS advances the application to KFS Issued.

15. Loan Agreement & Aadhaar eSign

The agreement renders the clauses and signature blocks, then `ESignService` (REST `/v1/esign/*`) initiates signing with the provider and records an `ESign_Request__c` with a SHA-256 document hash. On the signed callback it stores the signed PDF as a File, marks the document signed in the store, and advances the application to Agreement Signed.

```
// initiate: hash the rendered agreement, create ESign_Request__c (Initiated)
// callback (SIGNED): store signed PDF File -> DMSService.markSigned -> Loan_Application
'Agreement Signed'
```

Provider independence: `ESignService` is written against an abstract provider so the eSign vendor (an empanelled ESP such as Protean/CDAC, an ASP/aggregator such as Leegality or Digio, or eMudhra) is a configuration choice, not an architectural one. Where a vendor cannot be onboarded, the platform also ships its own electronic-signature framework (Section 15A).

15A. In-House eSign Framework (provider-independent)

15A.1 Legal positioning

Indian law recognises two broad categories under the Information Technology Act, 2000: digital signatures backed by a certificate from a CCA-licensed Certifying Authority (including Aadhaar eSign, which only a licensed CA/ESP can issue), and the wider class of electronic signatures whose validity

rests on Section 10A (electronic contracts) together with a reliable authentication method and an audit trail. A lender cannot self-issue Aadhaar eSign — that requires a CA licence — but it can operate its own electronic-signature service, and it can additionally apply an organizational PKI signature using a document-signer certificate it obtains from any licensed CA.

Regulatory caveat: for loan agreements executed on digital-lending platforms, current RBI digital-lending expectations point to IT Act-compliant digital signatures, and plain click-wrap / OTP-only signatures may not by themselves satisfy that bar. The in-house framework is therefore intended (a) for documents and consents where an electronic signature is sufficient, and (b) as the orchestration layer that pairs an OTP-authenticated, audited signing ceremony with either an empanelled Aadhaar-eSign provider or an organizational PKI (PAdES) seal for the loan agreement itself. This is engineering guidance, not legal advice; the signature strategy should be confirmed with counsel.

15A.2 Design

(REST /v1/inhouse-esign/*) issues and verifies a one-time password, records explicit consent and a tamper-evident audit trail, hashes the document (SHA-256), stores a drawn/typed signature image, and produces a Certificate of Completion through the DMS. Secrets are minimised: Salesforce stores only a salted hash of the OTP and discards it after signing; the plaintext OTP exists only transiently in MuleSoft while it is dispatched by SMS/email.

15A.3 Signing ceremony

1. Initiate: create the envelope, hash the document, issue an OTP; MuleSoft dispatches it to the signer's registered mobile/email (provider-abstract).
2. Verify: the signer submits the OTP and accepts the consent statement; the service validates the OTP (expiry + attempt limits), captures the signature image, the source IP and user-agent, and appends each step to the audit trail.
3. Finalize: for an electronic-only signature the request is marked Signed immediately; where an organizational PKI seal is configured, MuleSoft applies a PAdES signature with the lender's document-signer certificate and an RFC-3161 trusted timestamp, then posts the sealed PDF back.
4. Complete: the Certificate of Completion (signer, document hash, consent, audit trail, IP/agent, seal method) is stored in the DMS, the agreement is marked signed, and the loan advances to Agreement Signed.

OTP: salted SHA-256 hash stored; plaintext never persisted; expiry + max attempts
 Consent: explicit acceptance recorded with timestamp + IP in the audit trail
 Hash: SHA-256 of the exact document version binds the signature to the file
 Seal: optional PAdES (organizational DSC + TSA timestamp) -> tamper-evident PDF
 Output: Certificate of Completion (audit trail) filed in the DMS

15A.4 Endpoints

Endpoint	Purpose
POST /v1/inhouse-esign/initiate	Create envelope, hash document, issue OTP
POST /v1/inhouse-esign/verify-otp	Verify OTP + consent, capture signature & audit, finalize
POST /v1/inhouse-esign/seal-complete	Attach the organizational-PKI (PAdES) sealed PDF
GET /v1/inhouse-esign/status	Status + audit trail

16. NACH / eNACH Mandate

(REST /v1/nach/*) creates a Mandate__c (bank account, IFSC, max amount, frequency, debit day) and, on the provider callback, records the UMRN, sets the mandate Active and moves the application to Mandate Active. The mandate is later used for automated EMI debits (Section 20).

17. Document Management System (DMS)

Every generated document is registered in Generated_Document__c through DMSService, which inserts the ContentVersion + ContentDocumentLink and the index record in one operation, with versioning (a new Final supersedes the prior Final of the same type), retention, language, source system and tags. This is the single catalogue the borrower portal, agents and the RAG layer all read from.

17.1 Document types

KYC Report · Key Facts Statement · Loan Agreement · Sanction Letter · NACH Mandate · Insurance Proposal · Insurance COI · Welcome Letter · Foreclosure Letter.

17.2 DMS responsibilities

- Store: render-to-File, link to the parent record, and index in one transaction.
- Version: supersede the prior Final of the same type; keep history.
- Retain: stamp a retention-until date per document class.
- Sign: flip status to Signed and attach the signed artifact (eSign callback).
- List: return a borrower's or loan's documents for portal/agent/RAG use.

18. Insurance

Credit-life or related cover is captured in Insurance_Policy__c (insurer, policy type/number, sum assured, premium, dates) with the certificate of insurance rendered by InsuranceCOI and stored through the DMS.

PART E

Payments — PhonePe Merchant

19. Payment Capabilities

(REST /v1/phonepe/*) routes by URI to initiate or callback handling and records every attempt as a Payment_Transaction__c (type, amount, gateway, merchant & gateway transaction ids, instrument/UPI VPA, status, timestamps).

Capability	Type	Effect on success
EMI collection	Inbound collect	Mark earliest pending installment Paid; reduce outstanding; close loan at zero
Disbursement	Outbound payout	Record disbursed amount/date on the loan account
Refund	Reversal	Record a reversing transaction
Mandate debit	Scheduled	Auto-collect an EMI via the active NACH/eNACH mandate

20. Checksum, Callback & Reconciliation

Requests to PhonePe carry the X-VERIFY checksum (SHA-256 of the base64 payload + path + salt, with the salt index). The callback decodes the base64 response, maps PAYMENT_SUCCESS to COMPLETED, and processCallback applies the financial effect:

```
// EMI success: find earliest Repayment_Schedule (Pending) -> Paid,
// reduce Loan_Account outstanding, close loan if outstanding <= 0
// Disbursement success: set disbursed amount/date on the account
```

Note: PhonePe endpoints and the X-VERIFY scheme are modelled to the merchant pattern and must be checked against the current PhonePe PG documentation before production.

20.1 PhonePe sandbox — concrete mapping

The provider call MuleSoft makes against the UAT base

<https://api-preprod.phonepe.com/apis/pg-sandbox>. The JSON payload is base64-encoded and wrapped as { "request": "<base64>" }; the X-VERIFY header authenticates it:

```
X-VERIFY = SHA256( base64Payload + "/pg/v1/pay" + saltKey ) + "###" + saltIndex

// decoded pay payload (amount in paise)
{ "merchantId":"PGTESTPAYUAT", "merchantTransactionId":"EMI-1718273400",
  "amount":1499900, "redirectUrl":"https://portal/return", "redirectMode":"POST",
  "callbackUrl":"https://mule/v1/payments/callback",
  "paymentInstrument":{"type":"PAY_PAGE" } }

// sandbox response
{ "success":true, "code":"PAYMENT_INITIATED",
  "data":{"merchantTransactionId":"EMI-1718273400",
    "instrumentResponse":{"type":"PAY_PAGE",
      "redirectInfo":{"url":"https://mercury-uat.phonepe.com/transact/pay?
token=..." } } } }
```

20.2 Callback & status mapping

```
// webhook (decoded) -> Salesforce /v1/phonepe/callback
{ "code":"PAYMENT_SUCCESS",
  "data":{"merchantTransactionId":"EMI-...", "transactionId":"T...", "state":"COMPLETED" } }
```

```
PAYMENT_SUCCESS -> "COMPLETED" -> Payment_Transaction Status "Success" (instalment Paid)
PAYMENT_ERROR   -> "FAILED"       -> Status "Failed"
(other)          -> "PENDING"
```

```
// reconciliation
GET /pg/v1/status/{merchantId}/{merchantTransactionId}
X-VERIFY = SHA256( "/pg/v1/status/{mid}/{txn}" + saltKey ) + "###" + saltIndex
```

Disbursement payouts use PhonePe's payout/refund APIs with the same checksum scheme and their own credentials; the success callback sets the disbursed amount/date on the loan account. A runnable `api-curl-collection.sh` reproduces these calls.

PART F

Intelligence — RAG & MCP

21. Document RAG

The platform answers questions grounded strictly in the borrower's own documents.

DocumentRAGService (REST /v1/rag/*) indexes documents into Document_Knowledge_Chunk__c (chunk text, token count, embedding model, vector-store id) and, at query time, assembles the matched chunks into a grounded context with citations back to the source document.

21.1 Retrieval pipeline (MuleSoft)

```
ragIndexFlow: chunk -> embed -> vector upsert -> persist chunk rows (Indexed_For_RAG__c)
ragQueryFlow: embed question -> vector search -> fetch chunks from Salesforce
               -> Claude grounded answer (cite document + chunk)
```

Answers carry citations; if no chunk is relevant the system declines rather than speculating.

22. MCP Tool Surface

A Model Context Protocol server exposes a small, safe set of tools so an assistant (operations, credit, or borrower copilot) can act over the platform under least privilege.

Tool	Purpose	Mode
get_loan_status	Status & key terms of a loan	Read-only
get_repayment_schedule	Upcoming/paid installments	Read-only
list_documents	A borrower's/loan's documents	Read-only (metadata)
ask_documents	RAG answer grounded in documents	Read-only
get_portfolio_summary	Book-level servicing KPIs	Read-only
get_collections_queue	Upcoming & overdue instalments	Read-only
get_disbursement_queue	Loans awaiting payout	Read-only
ask_portfolio	Grounded answer over the loan book	Read-only
generate_document	Produce a KFS/agreement/etc.	Write — confirm-gated

22.1 Guardrails

- Read-only by default; the one write tool (generate_document) is confirmation-gated.
- PII redaction: Aadhaar and bank numbers exposed only as last-4.
- **No autonomous decisions:** underwriting, disbursement, refunds and mandate cancellation are deliberately not exposed as tools.
- Self-hosted behind the configured URL with a least-privilege integration user.

22.2 Tools by persona

Persona / copilot	Tools
Operations copilot	get_portfolio_summary, get_collections_queue, get_disbursement_queue, ask_portfolio, list_documents

Persona / copilot	Tools
Credit officer	get_loan_status, list_documents, ask_documents, generate_document (confirm)
Borrower copilot	get_loan_status, get_repayment_schedule, ask_documents

PART G

Cross-Cutting Concerns

23. Consolidated Data Model

#	Object	Purpose
1	KYC_Application__c	Central identity/KYC case + verification trail
2	Consent_Artifact__c	Audio & written consent records
3	Sarvam_Metadata__c	OCR / translation / photo metadata
4	Claude_Evaluation__c	AI adjudication verdict & narrative
5	Video_KYC_Session__c	Liveness / agent video sessions
6	Offline_KYC_Record__c	Aadhaar offline & field-agent capture
7	Loan_Application__c	Origination, underwriting, pricing
8	Loan_Account__c	Booked loan & servicing state
9	Repayment_Schedule__c	Amortised EMI installments
10	Payment_Transaction__c	PhonePe payments (all types)
11	Mandate__c	NACH / eNACH mandate
12	ESign_Request__c	Aadhaar eSign requests & hashes
13	Insurance_Policy__c	Credit-life / insurance cover
14	Generated_Document__c	DMS catalogue of all documents
15	Document_Knowledge_Chunk__c	RAG chunks & vector references

24. Security Model

- **Credential isolation:** Sarvam, Anthropic, Salesforce, PhonePe and eSign secrets live only in MuleSoft secured properties; Salesforce reaches MuleSoft through a Named Credential.
- **Least-privilege integration user:** a single permission set grants the API user access to the Apex services, the document pages, and CRUD/field-level security across the 15 objects (delete withheld).
- **PII minimisation:** Aadhaar and bank numbers are masked to last-4 in tool output and UI; full values stay in the record under sharing controls.
- **Document integrity:** agreements are hashed (SHA-256) before signing; signed artifacts are stored immutably and versioned.
- **Auditability:** AI narratives, consent, payments and document versions form a continuous audit trail on the application and loan.

25. API Contract Catalogue

Endpoint	Method	Purpose
/v1/headlessKYC	POST	Commit a KYC application (document channel)
/v1/videoKYC	POST	Commit a video-liveness session

Endpoint	Method	Purpose
/v1/offlineKYC	POST	Commit an Aadhaar-offline / field-agent record
/v1/phonepe/initiate	POST	Start a payment / disbursement
/v1/phonepe/callback	POST	PhonePe webhook -> apply financial effect
/v1/nach/create	POST	Create a NACH/eNACH mandate
/v1/nach/callback	POST	Mandate result -> UMRN / Active
/v1/esign/initiate	POST	Initiate agreement signing
/v1/esign/callback	POST	Signed -> store signed PDF & advance
/v1/inhouse-esign/initiate	POST	In-house eSign: envelope + OTP
/v1/inhouse-esign/verify-otp	POST	In-house eSign: verify + finalize
/v1/rag/index	POST	Index documents into knowledge chunks
/v1/rag/query	POST	Grounded answer with citations
/v1/portfolio/summary	GET	Book-level servicing KPIs
/v1/portfolio/collections	GET	Upcoming + overdue instalments
/v1/portfolio/disbursements	GET	Loans awaiting payout
/v1/agent/message	POST	Agent API broker: converse with the Lending Concierge
/einstein/ai-agent/v1/...	POST	Salesforce Agent API: sessions & messages

25.1 Representative payloads

```
// PhonePe EMI callback (decoded)
{ "merchantTransactionId":"EMI-...", "transactionId":"T...",
  "code":"PAYMENT_SUCCESS", "amount":["1499900"] } // paise

// RAG query
{ "loanApplicationId":"a00...", "question":"What is my EMI and due date?" }
// -> { "answer":"...", "citations":[{"documentType":"Key Facts Statement", "chunk":3}] }
```

26. Deployment

5. `sf org login web --alias platform-org`
6. `sf project deploy start --manifest manifest/package.xml --test-level RunLocalTests`
7. **Replace the Indic font:** upload a real Noto Sans Unicode TTF as the static resource (or enable Advanced PDF rendering).
8. **MuleSoft:** import the three flow files; configure Sarvam, Claude, PhonePe, eSign, embedding and vector-store request configs plus the UIDAI certificate.
9. **MCP:** host the MCP server behind the configured URL with the least-privilege integration user.
10. Assign the integration permission set to the API user.

27. Implementation Notes & Abstractions

The following are deliberately provider-abstract and must be wired to your chosen vendors before production:

- **PhonePe:** endpoints and the X-VERIFY checksum follow the merchant pattern; verify against current PhonePe PG documentation.
- **eSign / eNACH:** the provider (e.g. Protean/CDAC) request/response shape is abstracted.

- **eSign legal boundary:** Aadhaar eSign can only be issued by a CCA-licensed CA/ESP and cannot be self-built; for RBI digital-lending loan agreements an IT Act-compliant digital signature is expected. The in-house framework (Section 15A) covers documents where an electronic signature suffices and pairs OTP-authenticated, audited signing with either an empanelled Aadhaar-eSign provider/aggregator or an organizational PKI (PAdES) seal. Confirm the signature strategy with legal counsel.
- **RAG infrastructure:** the embedding model and vector store (e.g. Pinecone / pgvector) are provider-abstract.
- **Aadhaar offline:** share-code unzip and UIDAI XML signature verification need a real Java/Mule component and the current UIDAI certificate.
- **Video liveness:** /v1/face-verify is provider-abstract; point it at your biometric service.
- **Indic PDF fonts:** ship a real Noto TTF static resource or enable Advanced PDF rendering; the placeholder renders Latin only.
- **AI autonomy:** underwriting, disbursement and refunds are intentionally not exposed as autonomous MCP tools.
- **Auth hardening:** replace any static Salesforce session token with an OAuth client-credentials sub-flow; add retry/DLQ on the integration flows.

PART H

Agentforce — Agentic Layer

28. Lending Concierge Agent

The agentic layer adds a natural-language front door over the same services. The **Lending Concierge** is an Agentforce agent: a Bot shell whose reasoning engine is a `GenAiPlannerBundle` (the Atlas reasoning engine). The planner orchestrates a set of **subagents** (`GenAiPlugin`), each owning a job-to-be-done; every **action** (`GenAiFunction`) binds to an `@InvocableMethod` on `AgentforceLendingActions`, which is a thin wrapper over the existing platform services. The agent reasons; the proven services do the work.

```

Bot: Lending Concierge
├─ GenAiPlannerBundle: Lending_Concierge_Planner (Atlas / ReAct)
│   ├── GenAiPlugin: Onboarding → Get KYC Status
│   ├── GenAiPlugin: Origination → Get Loan Status, Get Underwriting Summary,
│   │   Generate Loan Document*
│   ├── GenAiPlugin: Servicing → Get Portfolio Summary, Get Collections, Get
│   │   Repayment Schedule
│   ├── GenAiPlugin: Documents & eSign → Ask Documents, Prepare eSign*
│   └── GenAiPlugin: Payments → Request Disbursement Approval*
(* = write action, confirm-gated)    GenAiFunction → AgentforceLendingActions (Apex)
→ platform service

```

28.1 Subagent catalogue

Subagent (GenAiPlugin)	Scope	Actions
Onboarding	Identity / KYC state before lending	Get KYC Status
Origination	Application status, underwriting, document generation	Get Loan Status; Get Underwriting Summary; Generate Loan Document*
Servicing	Operations: KPIs, collections, schedules	Get Portfolio Summary; Get Collections; Get Repayment Schedule
Documents & eSign	Grounded answers + start signing	Ask Documents; Prepare eSign*
Payments	Disbursement requests	Request Disbursement Approval*

28.2 Action inventory

Each action maps to one invocable Apex method and one underlying service. Read actions run autonomously; write actions are confirm-gated.

Action (GenAiFunction)	Invocable Apex	Underlying service	Mode
Get KYC Status	getKycStatus	KYC_Application__c	Read
Get Loan Status	getLoanStatus	Loan_Application__c + Loan_Account__c	Read
Get Underwriting Summary	getUnderwritingSummary	Loan_Application__c	Read
Generate Loan Document	generateLoanDocument	LoanDocumentService	Write*
Get Portfolio Summary	getPortfolioSummary	LoanServicingService	Read
Get Collections	getCollections	LoanServicingService	Read
Get Repayment Schedule	getRepaymentSchedule	Repayment_Schedule__c	Read
Ask Documents	askDocuments	Document_Knowledge_Chunk__c (RAG)	Read

Action (GenAiFunction)	Invocable Apex	Underlying service	Mode
Prepare eSign	prepareESign	InHouseESignService	Write*
Request Disbursement Approval	requestDisbursementApproval	Payment_Transaction__c	Write*

29. Agent Guardrails

- Read actions run autonomously; write actions (generate document, prepare eSign, request disbursement approval) are confirmation-gated in the topic instructions.
- **No autonomous money movement:** Request Disbursement Approval only queues a `Payment_Transaction__c` in Initiated for a human to approve and execute; the agent never disburses, refunds, or cancels a mandate.
- Underwriting is read-only to the agent — it can explain a decision but not make or change one.
- Ask Documents answers only from filed documents and returns citations; if nothing matches it says so rather than speculating.
- Actions inherit platform security (sharing, USER_MODE, FLS); Aadhaar and bank numbers are surfaced only as last-4.
- The agent composes with the RAG service (grounding) and complements the MCP tool surface (Section 22): MCP exposes tools to external copilots, while Agentforce runs the in-platform conversational agent over the same Apex.

30. Agent API Access in Headless 360

Any channel — the Experience Cloud portal, the mobile app, or an external service — converses with the agent through the Salesforce **Agent API**. Authentication is OAuth client-credentials via an External Client App (ECA). The Agent API is not supported for the “Agentforce (Default)” type; the Lending Concierge must be a custom, activated agent. The `agentId` is the Bot Id (prefix 0Xx).

Step	Call
Token	POST {myDomain}/services/oauth2/token (grant_type=client_credentials)
Start session	POST {api}/einstein/ai-agent/v1/agents/{agentId}/sessions
Send message	POST {api}/einstein/ai-agent/v1/sessions/{sessionId}/messages
End session	DELETE {api}/einstein/ai-agent/v1/sessions/{sessionId}

The platform also ships a MuleSoft broker so callers need not handle tokens or session lifecycle directly: POST `/v1/agent/message` obtains a token, starts a session if needed, sends the message and returns the reply with a session id; DELETE `/v1/agent/session` ends it. Synchronous and streaming (SSE) modes are both available on the Agent API.

31. Agentforce Deployment Notes

- **Metadata types:** GenAiPlannerBundle (API v64+, replaces GenAiPlanner), GenAiPlugin (subagent/topic), GenAiFunction (action, with input/output schema.json), and Bot + BotVersion.

- **Dependency order:** deploy the Apex actions and GenAiFunctions first, then GenAiPlugins, then the GenAiPlannerBundle, then Bot/BotVersion — a planner referencing a missing topic or action fails to deploy.
- **Binding:** each GenAiFunction's `invocationTarget / invocationTargetType=apex` must resolve to a deployed invocable action; `isConfirmationRequired=true` marks the write actions.
- The deployable, tested core is the Apex action layer (AgentforceLendingActions + test). The agent metadata here is a representative scaffold aligned to the metadata API; finalise topics, instructions and assignments in Agent Builder and adjust to your org's API version (Winter '26 isolates topics/actions per planner version).
- Before using the Agent API: activate the agent and create the ECA (client-credentials); configure the MuleSoft secure properties for the broker.